

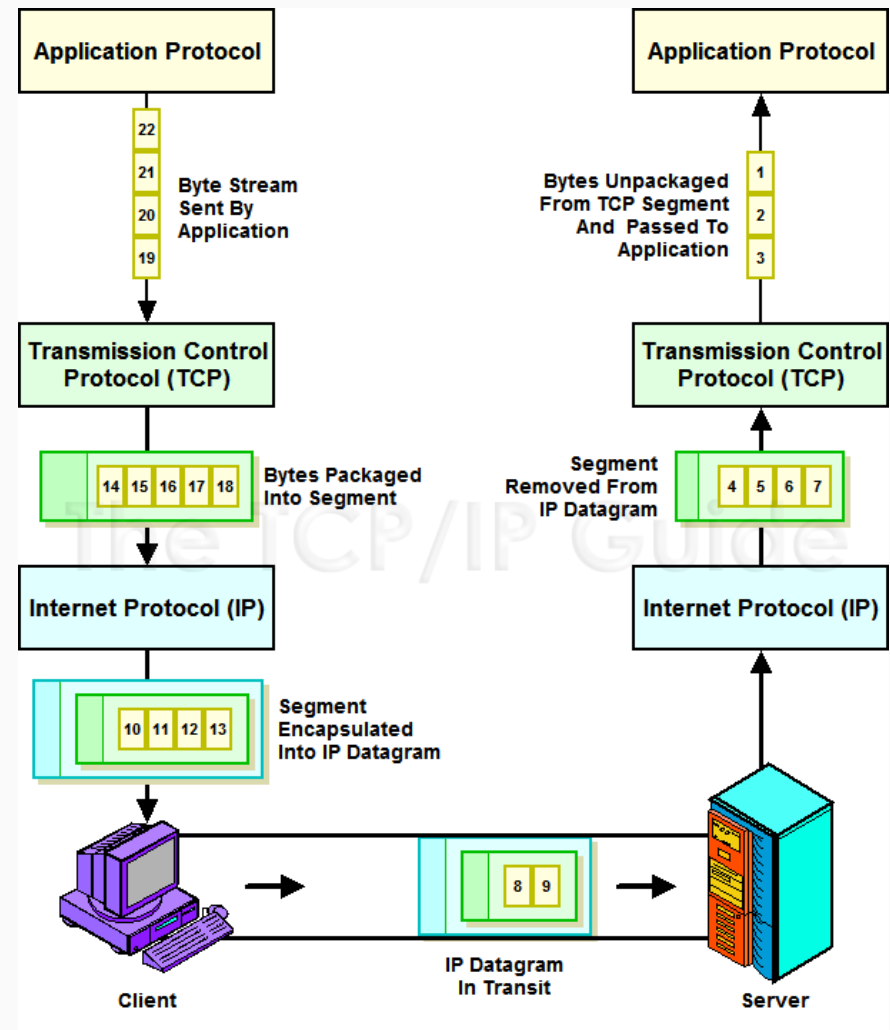
Introduction to MQTT

Marco Zennaro, PhD

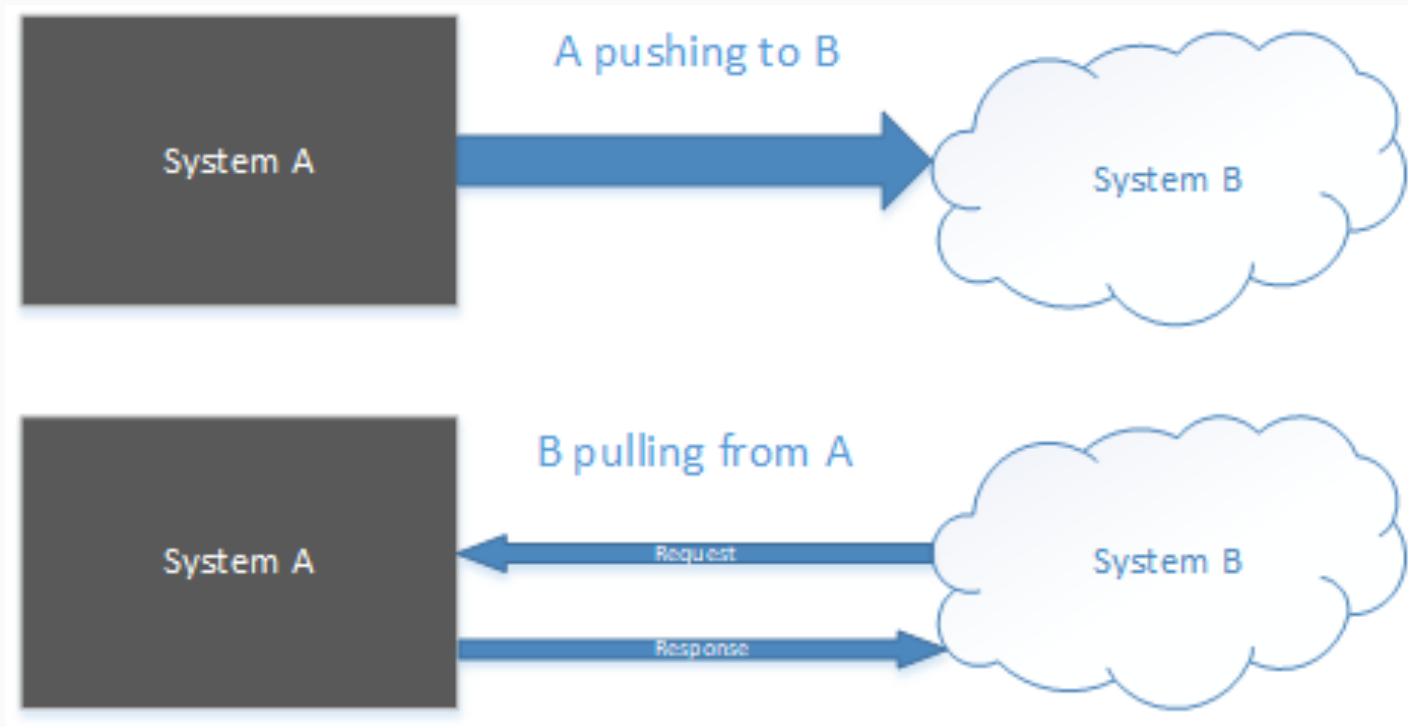
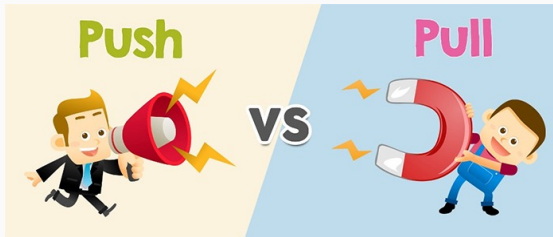
The Abdus Salam International Centre for Theoretical Physics –Italy

From “byte streams” to “messages”

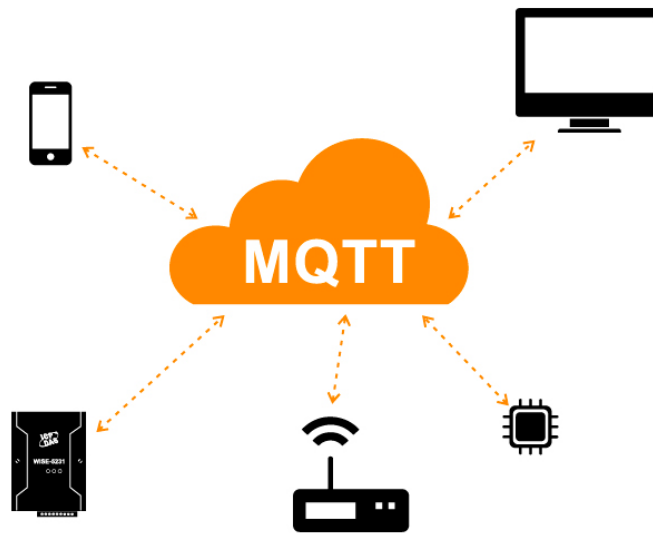
- The “old” vision of data communication was based on **reliable byte streams**, i.e., TCP
- Nowadays **messages interchange** is becoming more common
 - E.g., Twitter, Whatsapp, Instagram, Snapchat, Facebook,...
- Actually is not that new...
 - emails: SMTP+MIME,
 - FTP,



Ways to interchange “messages”

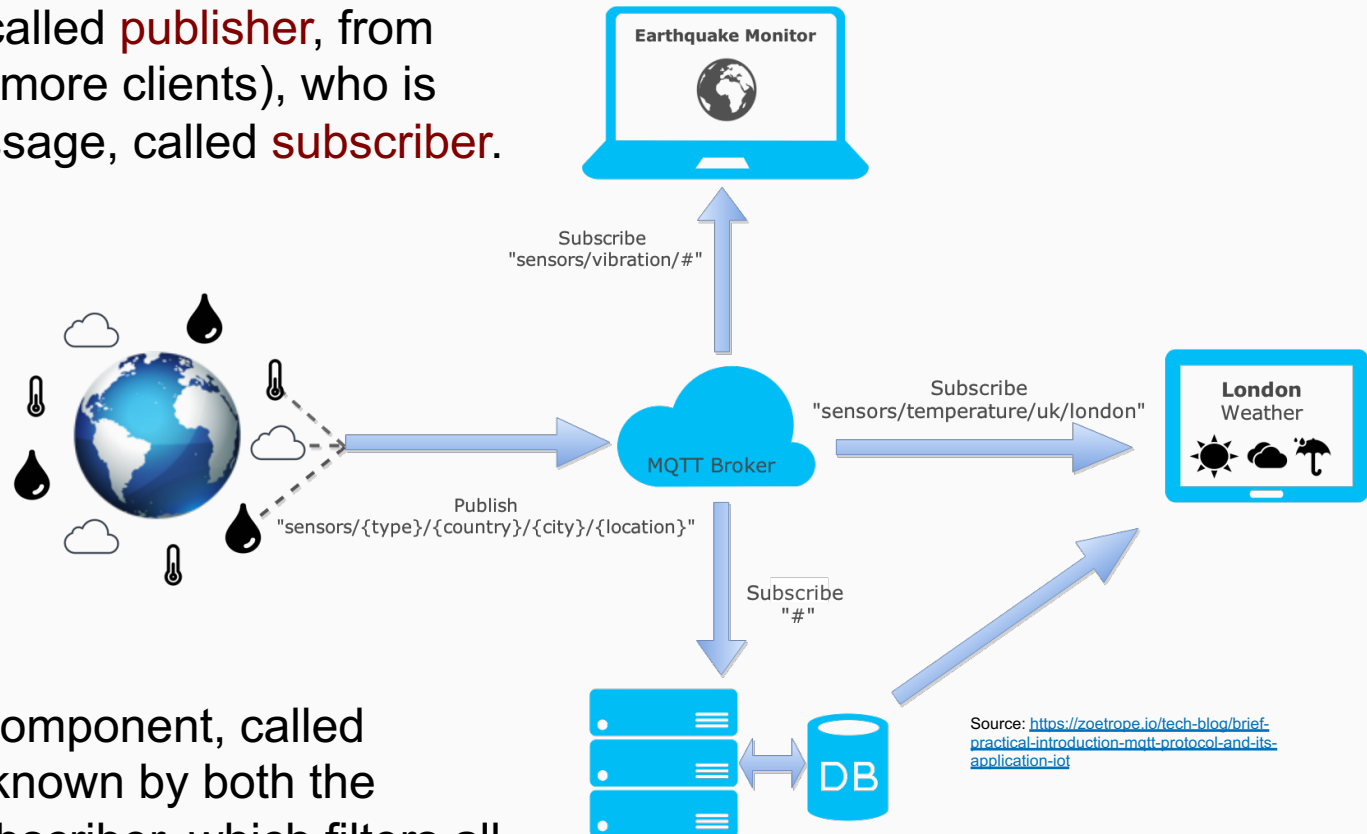


MQTT



Pub/sub approach: an example

- Pub/Sub separate a client (or more clients), who is sending a message about a specific **topic**, called **publisher**, from another client (or more clients), who is receiving the message, called **subscriber**.



- There is a third component, called **broker**, which is known by both the publisher and subscriber, which filters all incoming messages and distributes them accordingly.

Pub/sub approach: a growing approach

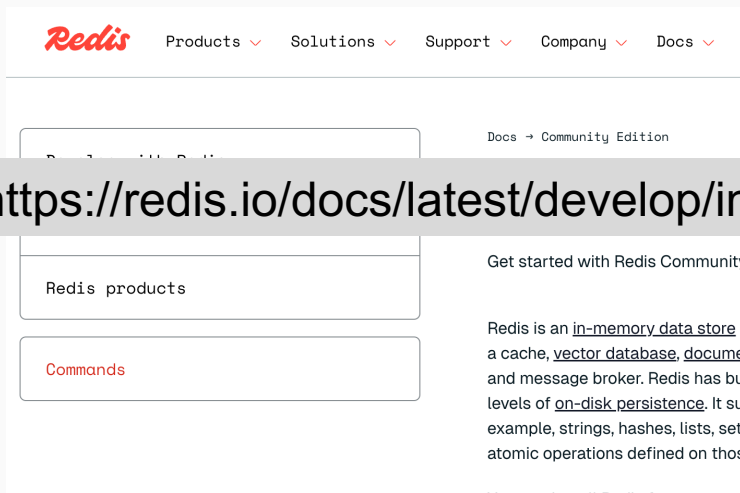


<https://aws.amazon.com/sns/>

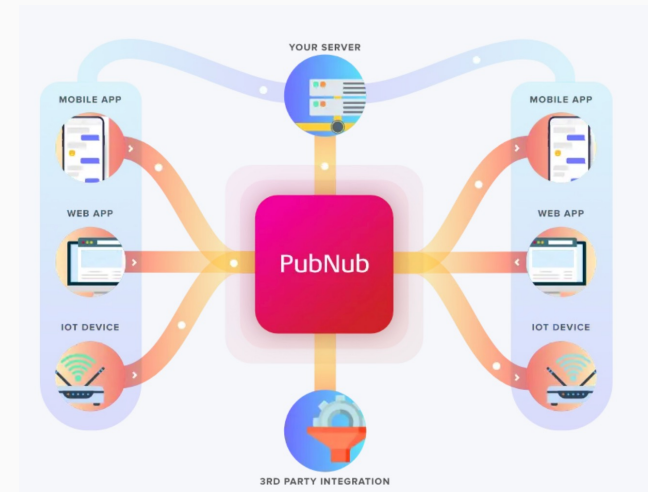
Google Cloud

Cloud Pub/Sub

<https://cloud.google.com/pubsub/>



<https://redis.io/docs/latest/develop/interact/pubsub/>



<https://www.pubnub.com/>

Message Queuing Telemetry Transport

- A **lightweight publish-subscribe protocol** that can run on embedded devices and mobile platforms → <http://mqtt.org/>
 - Low power usage.
 - Binary compressed headers
 - Maximum message size of 256MB
 - not really designed for sending large amounts of data
 - better at a high volume of low size messages.
- Documentation sources:
 - The MQTT community wiki:
 - <https://github.com/mqtt/mqtt.org/wiki>
 - A very good tutorial:
 - <https://www.hivemq.com/mqtt/>



Some details about versions

- **MQTT 3.1.1 is the current version of the protocol.**
 - Standard document here:
 - <http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html>
 - October 29th 2014: MQTT was officially approved as OASIS Standard.
 - https://www.oasis-open.org/committees/tc_home.php?wg_abbrev=mqtt
- MQTT v5.0 is the successor of MQTT 3.1.1
 - Current status: Committee Specification 02 (7 March 2019)
 - <http://docs.oasis-open.org/mqtt/mqtt/v5.0/cs02/mqtt-v5.0-cs02.html>
 - **Not backward compatible**; too many new things are introduced so existing implementations have to be revisited, for example:
 - 1. More extensibility → user properties
 - 2. Improved error reporting (Reason Code & Reason String)
 - 3. Performance improvements and improved support for small clients
 - shared subscriptions
 - topic alias
 - 4. Formalized common patterns → payload format description
 - 5. Improved authentication

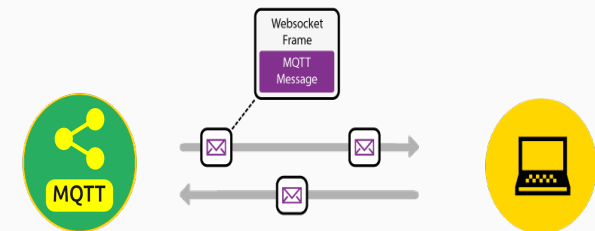
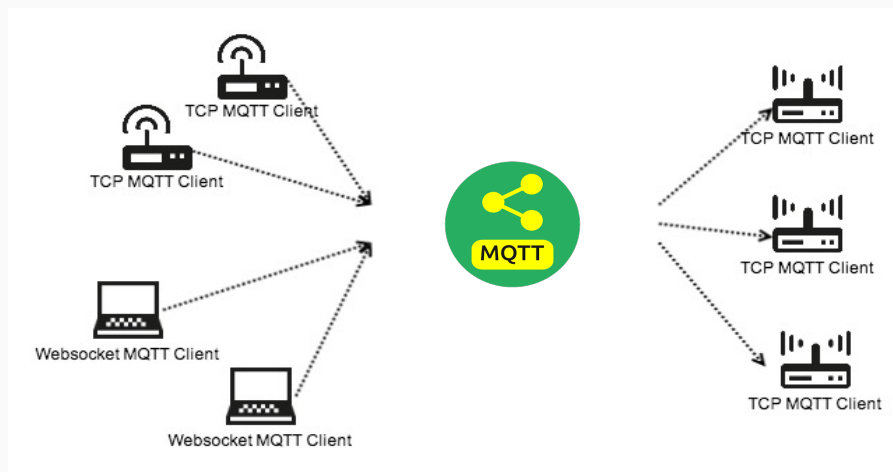
Distinction: MQTT & Message Queues

- There is some confusion about the name MQTT and whether the protocol is implemented as a Message Queue or not.
- It's useful to understand the differences between MQTT and a traditional message queue:
 - In a message queue, each incoming message is stored in the queue until it is picked up by a client (often called a consumer). If no client picks up the message, the message remains in the queue and waits to be consumed. In a message queue, it is not possible for a message not to be processed by any client, as it is in MQTT if nobody subscribes to a topic.
 - In a traditional message queue, a message can be processed by one consumer only. In MQTT the behavior is quite the opposite: every subscriber that subscribes to the topic gets the message.
 - Queues are named and must be created explicitly. A queue is far more rigid than a topic. Before a queue can be used, the queue must be created explicitly with a separate command. In contrast, MQTT topics are extremely flexible and can be created on the fly.
- More classical Message Queueing systems:
 - <https://kafka.apache.org/>
 - <https://zeromq.org/>



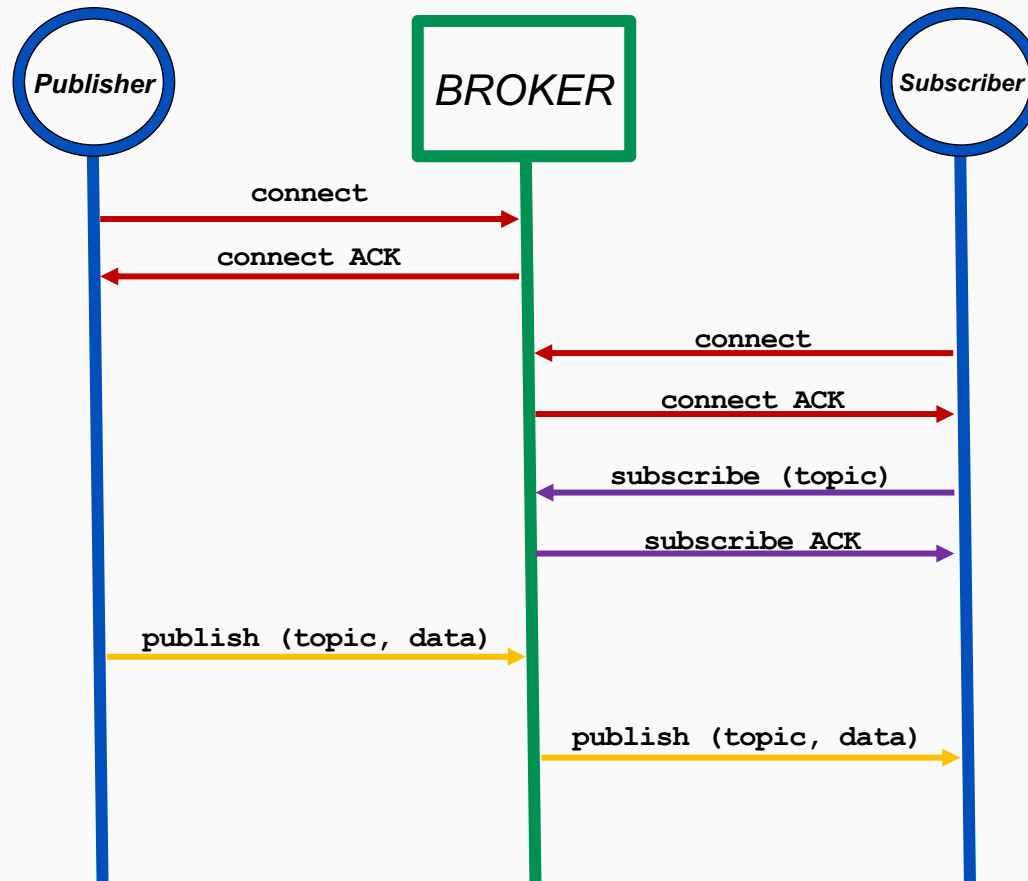
MQTT works on top of...

- mainly of TCP
 - There is also the closely related [MQTT for Sensor Networks \(MQTT-SN\)](#) where TCP is replaced by UDP → TCP stack is too complex for WSN
- websockets can be used, too!
 - Websockets allows you to receive MQTT data directly into a web browser.



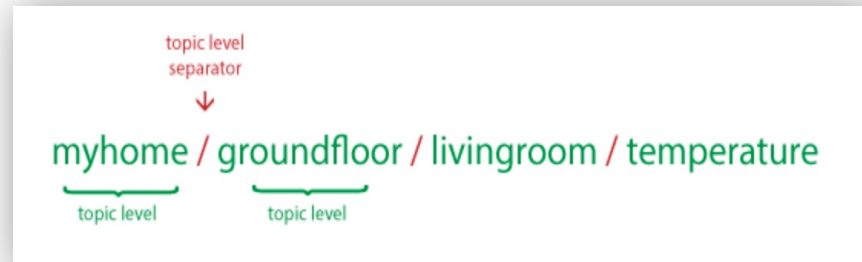
- Both, TCP & websockets can work on top of “Transport Layer Security (TLS)” (and its predecessor, Secure Sockets Layer (SSL))

Publish/subscribe interactions sequence



Topics

- MQTT Topics are structured in a hierarchy similar to folders and files in a file system using the forward slash (/) as a delimiter.
- Allow to create a user friendly and self descriptive **naming structures**
- Topic names are:
 - Case sensitive
 - use UTF-8 strings.
 - Must consist of at least one character to be valid.
- Except for the \$SYS topic **there is no default or standard topic structure.**



Special \$SYS/ topics

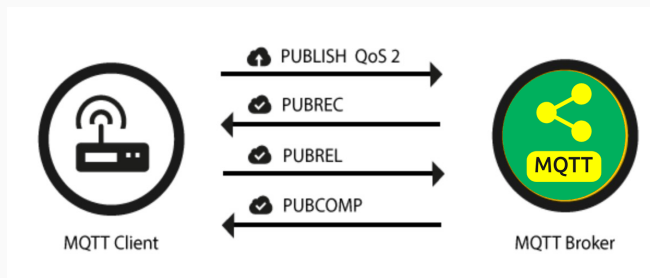
- \$SYS/broker/clients/connected
- \$SYS/broker/clients/disconnected
- \$SYS/broker/clients/total
- \$SYS/broker/messages/sent
- \$SYS/broker/uptime

Topics wildcards

- Topic subscriptions can have wildcards. These enable nodes to subscribe to groups of topics that don't exist yet, allowing greater flexibility in the network's messaging structure.
 - '+' matches anything at a given tree level
 - '#' matches a whole sub-tree
- Examples:
 - Subscribing to topic `house/#` covers:
 - `house/room1/main-light`
 - `house/room1/alarm`
 - `house/garage/main-light`
 - `house/main-door`
 - Subscribing to topic `house/+/main-light` covers:
 - `house/room1/main-light`
 - `house/room2/main-light`
 - `house/garage/main-light`
 - but doesn't cover
 - `house/room1/side-light`
 - `house/room2/side-light`

Quality of Service (QoS)

- Messages are published with a **Quality of Service (QoS)** level, which specifies delivery requirements.
- A **QoS 0 (“at most once”)** message is fire-and-forget.
 - For example, a notification from a doorbell may only matter when immediately delivered.
- With **QoS 1 (“at least once”)**, the broker stores messages on disk and retries until clients have acknowledged their delivery.
 - (Possibly with duplicates.) It’s usually worth ensuring error messages are delivered, even with a delay.
- **QoS 2 (“exactly once”)** messages have a second acknowledgement round-trip, to ensure that **non-idempotent messages** can be delivered exactly once.



Retained Messages!!!

- A retained message is a normal MQTT message with the **retained flag set to true**. The broker will store the last retained message and the corresponding QoS for that topic
 - Each client that subscribes to a topic pattern, which matches the topic of the retained message, will receive the message immediately after subscribing.
 - For each topic **only one retained message** will be stored by the broker.
- Retained messages can help newly subscribed clients to get a status update immediately after subscribing to a topic and don't have to wait until a publishing clients send the next update.
 - In other words, a retained message on a topic is the last known good value, because it doesn't have to be the last value, but it certainly is the last message with the retained flag set to true.

MQTT Keep alive

- The keep alive functionality assures that the connection is still open and both broker and client are connected to one another.
- The client specifies a time interval in seconds and communicates it to the broker during the establishment of the connection.
 - The interval is the longest possible period of time which broker and client can endure without sending a message.
 - If the broker doesn't receive a PINGREQ or any other packet from a particular client, it will close the connection and send out the last will and testament message (if the client had specified one).
- Good to Know
 - The MQTT client is responsible of setting the right keep alive value.
 - The maximum keep alive is 18h 12min 15 sec.
 - If the keep alive interval is set to **0**, the keep alive mechanism is deactivated.

“Will” message

- When clients connect, they can specify an optional “will” message, to be delivered if they are unexpectedly disconnected from the network.
 - (In the absence of other activity, a 2-byte ping message is sent to clients at a configurable interval.)
- This “last will and testament” can be used to notify other parts of the system that a node has gone down.

MQTT-Packet:	
CONNECT	
	
contains:	Example
clientId	"client-1"
cleanSession	true
username (optional)	"hans"
password (optional)	"letmein"
lastWillTopic (optional)	"/hans/will"
lastWillQos (optional)	2
lastWillMessage (optional)	"unexpected exit"
lastWillRetain (optional)	false
keepAlive	60

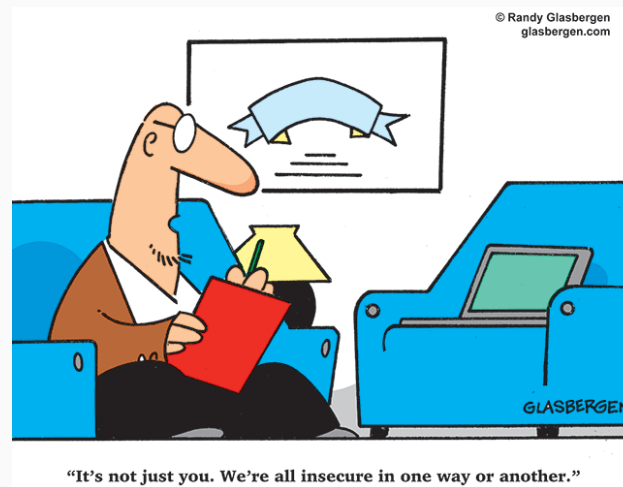
WHEN?

Persistent Session

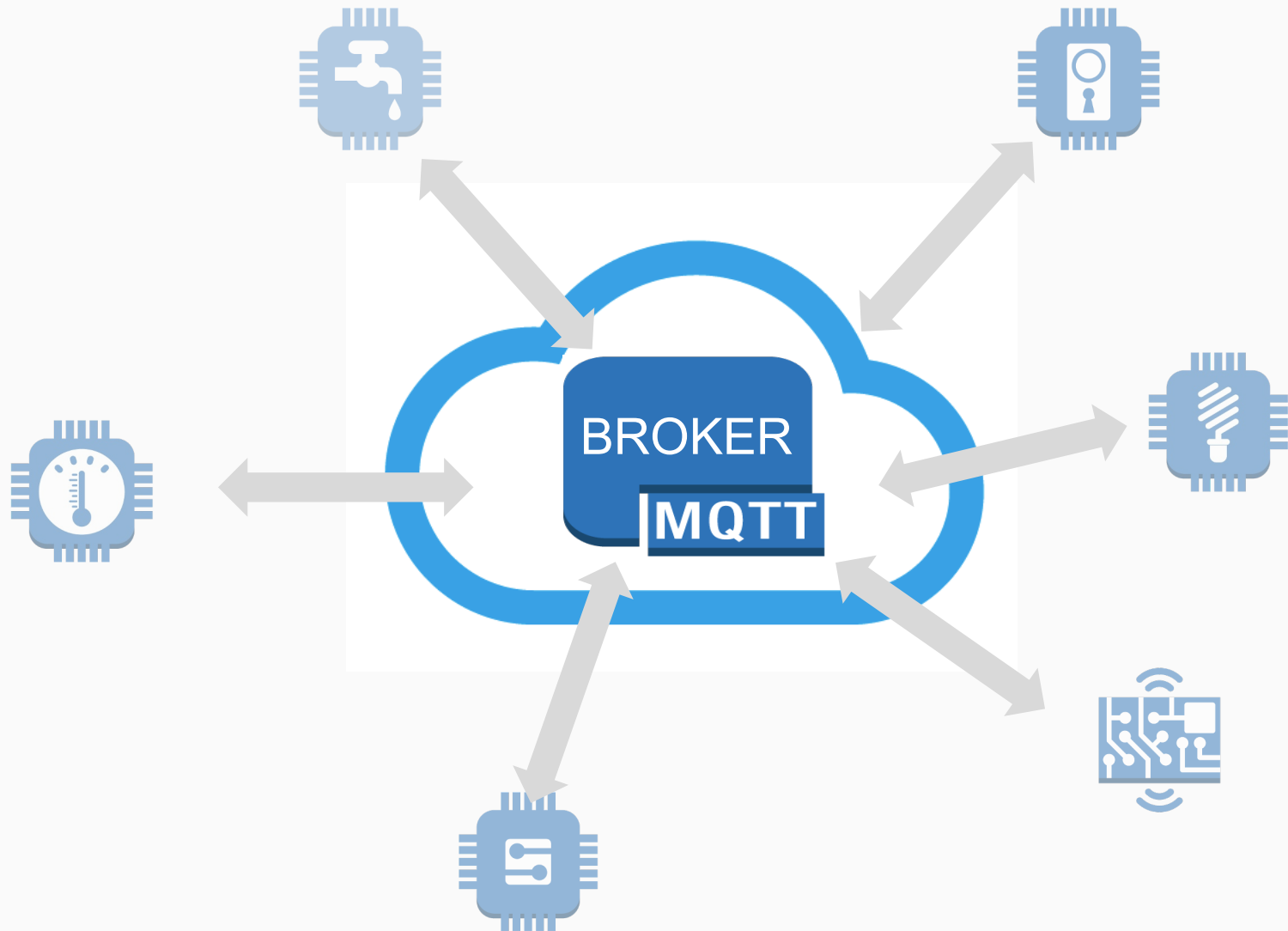
- To receive messages from an MQTT broker, a client connects to the broker and creates subscriptions to the topics in which it is interested. If the connection between the client and broker is interrupted during a non-persistent session, these topics are lost and the client needs to subscribe again on reconnect.
- Re-subscribing every time the connection is interrupted is a burden for constrained clients with limited resources.
- To avoid this problem, the client can request a persistent session when it connects to the broker. Persistent sessions save all information that is relevant for the client on the broker. The clientId that the client provides when it establishes connection to the broker identifies the session.
- In a persistent session, the broker stores the following information (even if the client is offline). When the client reconnects the information is available immediately.
 - Existence of a session (even if there are no subscriptions).
 - All the subscriptions of the client.
 - All messages in a Quality of Service (QoS) 1 or 2 flow that the client has not yet confirmed.
 - All new QoS 1 or 2 messages that the client missed while offline.
 - All QoS 2 messages received from the client that are not yet completely acknowledged.

A few words on security

- MQTT has the option for Transport Layer Security (TLS) encryption.
- MQTT also provides username/password authentication with the broker.
 - Note that the password is transmitted in clear text. Thus, be sure to use TLS encryption if you are using authentication.



Creating a broker

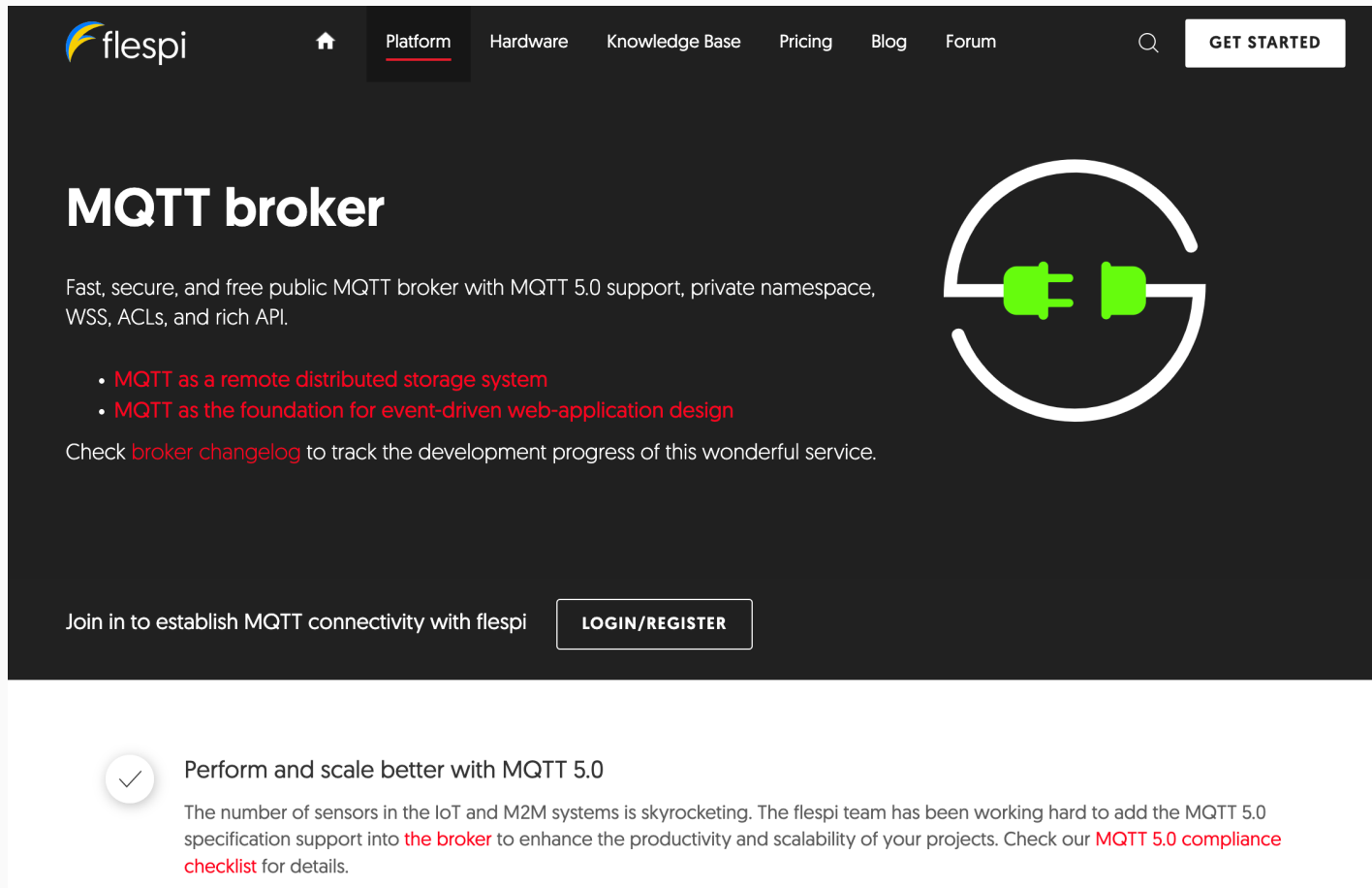


Available MQTT brokers

- The most widely used are:
 - <http://mosquitto.org/>
 - man page: <https://mosquitto.org/man/mosquitto-8.html>
 - <http://www.hivemq.com/>
 - The standard trial version only supports 25 connections.
- And also:
 - <https://www.rabbitmq.com/docs/mqtt>
 - <https://activemq.apache.org/components/classic/documentation/mqtt>
- A quite complete list can be found here:
 - <https://github.com/mqtt/mqtt.github.io/wiki/servers>

Cloud based brokers: flespi

<https://flespi.com/mqtt-broker>



The screenshot shows the website for flespi's MQTT broker. The header features the flespi logo, a home icon, and navigation links for Platform, Hardware, Knowledge Base, Pricing, Blog, and Forum. A search icon and a 'GET STARTED' button are also present. The main section is titled 'MQTT broker' and describes it as a fast, secure, and free public MQTT broker with MQTT 5.0 support, private namespace, WSS, ACLs, and a rich API. It lists two key features: 'MQTT as a remote distributed storage system' and 'MQTT as the foundation for event-driven web-application design'. A link to the 'broker changelog' is provided to track development progress. At the bottom of the main section, there is a call to action to 'Join in to establish MQTT connectivity with flespi' and a 'LOGIN/REGISTER' button. Below this, a section titled 'Perform and scale better with MQTT 5.0' explains that the flespi team has added MQTT 5.0 specification support to 'the broker' to enhance productivity and scalability, with a link to an 'MQTT 5.0 compliance checklist' for details.

flespi

Platform Hardware Knowledge Base Pricing Blog Forum

GET STARTED

MQTT broker

Fast, secure, and free public MQTT broker with MQTT 5.0 support, private namespace, WSS, ACLs, and rich API.

- MQTT as a remote distributed storage system
- MQTT as the foundation for event-driven web-application design

Check [broker changelog](#) to track the development progress of this wonderful service.

Join in to establish MQTT connectivity with flespi

LOGIN/REGISTER

Perform and scale better with MQTT 5.0

The number of sensors in the IoT and M2M systems is skyrocketing. The flespi team has been working hard to add the MQTT 5.0 specification support into [the broker](#) to enhance the productivity and scalability of your projects. Check our [MQTT 5.0 compliance checklist](#) for details.

Cloud based brokers: flespi

Free

€0/mo

For testing & development purposes. Note that limits are not extendable.

Included

10 devices ⓘ

2 channels ⓘ

1 stream ⓘ

Storage is not extendable ⓘ

<https://flespi.io/#/panel/list/sessions>

The screenshot displays the flespi web interface. The left sidebar shows the user profile 'Pietro manzoni@ieee.org' and a navigation menu. The main panel, titled 'Sessions', lists active MQTT sessions. Two sessions are shown: 'panel-4.55.19-c097c7f9' with 13 subscriptions and 'helpbox-3.2.1-d52b62d3' with 7 subscriptions. Each session entry includes a session ID, name, MQTT version, status (clean), and a size indicator.

<https://flespi.com/mqtt-api>

flespi MQTT broker connection details

- **Host** — `mqtt.flespi.io`.
- **Port** — **8883 (SSL)** or **1883 (non-SSL)**; for MQTT over WebSockets: **443 (SSL)** or **80 (non-SSL)**.
- **Authorization** — use a **flespi platform token** as MQTT session username; no password.
- **Client ID** — use any unique identifier within your flespi user session.
- **Topic** — you can publish messages to any topic except **flespi/**.
- **ACL** — both **flespi/** and **MQTT pub/sub** restrictions **determined by the token**.

I1RKMMIUJpplQoSgAQ8M.....JgijO1S1gt5rajaeIOaiaKWwlHt2z1z

Open brokers (“Sandboxes”)

■ TCP based:

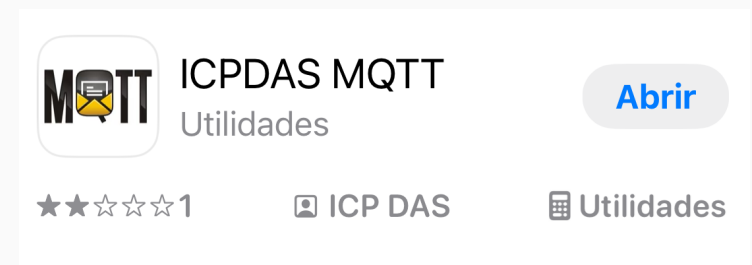
- <http://test.mosquitto.org/>
 - Hostname: **test.mosquitto.org**
- <https://www.hivemq.com/mqtt/public-mqtt-broker/>
 - Hostname: **broker.hivemq.com**
 - <https://www.mqtt-dashboard.com/>
- <https://iot.eclipse.org/projects/sandboxes/>
 - Hostname: **mqtt.eclipseprojects.io**
- Ports:
 - standard: 1883
 - encrypted: 8883 (*TLS v1.2, v1.1 or v1.0 with x509 certificates*)

■ Websockets based:

- broker.mqttdashboard.com port: 8000
- test.mosquitto.org port: 8080
- broker.hivemq.com port: 8000

■ https://github.com/mqtt/mqtt.org/wiki/public_brokers

MQTT clients: iOS



MQTT clients: Android



MQTT Dash (IoT, Sn)
Routix software

★★★★★



MyMQTT
instant:solutions OG

★★★★★



IoT MQTT Panel
Rahul Kundu

★★★★★



IoT MQTT Dashboard
Nghia TH

★★★★★



MQTT Client
Webneurons

★★★★★



MQTT Snooper
Maxime Carrier

★★★★★



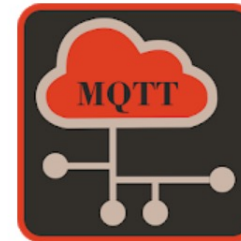
MQTIZER - Free MQ
Sanyam Arya

★★★★★



Linear MQTT Dashboard
ravendmaster

★★★★★



Virtuino MQTT
Ilias Lamprou

★★★★★



Mqtt Client
Darlei Kroth

★★★★★

MQTT websocket clients

<http://test.mosquitto.org/ws.html>

MQTT over WebSockets

This is a very early/incomplete/broken example of MQTT over Websockets for test.mosquitto.org. Play around with the buttons below, but don't be surprised if it breaks or isn't very pretty. If you want to develop your own websockets/mqtt app, use the url `ws://test.mosquitto.org/mqtt`, use subprotocol "mqtt" (preferred) or "mqttv3.1" (legacy) and binary data. Then just treat the websocket as a normal socket connection and read/write MQTT packets.

Usage

Click Connect, then use the Publish and/or Subscribe buttons. You should see text appear below. If you've got another mqtt client available, try subscribe to a topic here then use your other client to send a message to that topic.

Broker

Publish

Connect Disconnect

Topic:

Payload:

Publish

Topic: \$SYS/#

Subscribe Unsubscribe

<http://mitsuruog.github.io/what-mqtt/>

MQTT on Websocket sample

message clear

Connect / Disconnect

connect disconnect

MQTT broker on websocket

Address:

Subscribe / Unsubscribe

Topic:

subscribe unsubscribe

Publish

Topic:

HIVEMQ Websockets Client Showcase

Subscribe/Unsubscribe

HIVEMQ CLOUD Need a fully managed MQTT broker? Get your own Cloud broker and connect up to 100 devices for free. [Get your free account](#)

Connection

Host Port ClientID [Connect](#)

Username Password Keep Alive SSL ☒ Clean Session ☒

Last-Will Topic Last-Will QoS Last-Will Retain ☐

Last-Will Message

Publish **Subscriptions** **Messages**

<http://www.hivemq.com/demos/websocket-client/>



MQTT Explorer

An all-round MQTT client that provides a structured topic overview

MQTT in Python

- Source
 - <https://github.com/eclipse-paho/paho.mqtt.python>

- The Python client can be downloaded and installed from [PyPI](https://pypi.org/) using the pip tool:

```
pip install paho-mqtt
```

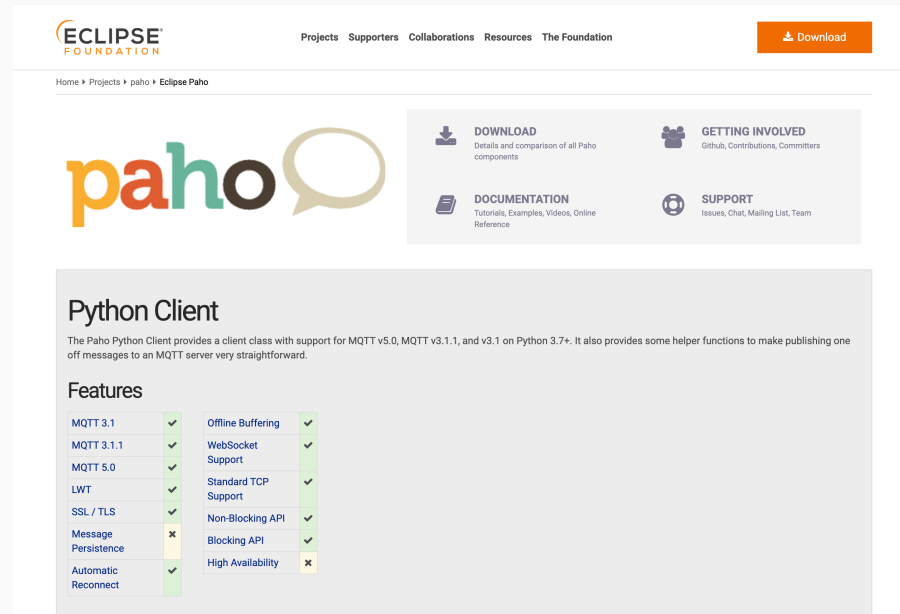
or

```
python3 -m pip install paho-mqtt
```

- Documentation

<https://eclipse.dev/paho/files/paho.mqtt.python/html/index.html>

<https://eclipse.dev/paho/index.php?page=downloads.php>



The screenshot shows the Eclipse Paho website. At the top, there's a navigation bar with links: Projects, Supporters, Collaborations, Resources, The Foundation, and a Download button. Below the navigation bar, the Paho logo is displayed. To the right of the logo are four sections: DOWNLOAD (Details and comparison of all Paho components), GETTING INVOLVED (Github, Contributions, Committers), DOCUMENTATION (Tutorials, Examples, Videos, Online Reference), and SUPPORT (Issues, Chat, Mailing List, Team). The main content area is titled "Python Client" and describes the Paho Python Client's capabilities. Below the description is a "Features" table.

Feature	Status	Feature	Status
MQTT 3.1	✓	Offline Buffering	✓
MQTT 3.1.1	✓	WebSocket Support	✓
MQTT 5.0	✓	Standard TCP Support	✓
LWT	✓	Non-Blocking API	✓
SSL / TLS	✓	Blocking API	✓
Message Persistence	✗	High Availability	✗
Automatic Reconnect	✓		

Python example 1: the simplest subscriber

```
# File: sisub200.py
#
# The simplest MQTT subscriber using paho-mqtt (version 2.0.0)

import paho.mqtt.client as mqtt

# Configuration
THE_BROKER = "test.mosquitto.org"
THE_TOPIC = "$SYS/#"

# Callback function triggered when the client receives a CONNACK
response from the broker.
def on_connect(mqttc, obj, flags, reason_code, properties=None):
    print(f"Connected to {mqttc._host}:{mqttc._port}")
    print(f"Flags: {flags}")
    print(f"Reason Code: {reason_code}")
    print(f"Properties: {properties}")

    mqttc.subscribe(THE_TOPIC, qos=0)

# Callback function triggered when the client receives a message
from the broker.
def on_message(mqttc, obj, msg):
    print(f"{msg.topic} {msg.qos} {msg.payload}")

# Callback function triggered when a subscription is made.
def on_subscribe(mqttc, obj, mid, reason_code_list,
properties=None):
    print(f"Subscribed: {mid} {reason_code_list}")
```

```
if __name__ == "__main__":
    # Create a new MQTT client instance
    client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2,
        client_id="",
        clean_session=True,
        userdata=None,
        protocol=mqtt.MQTTv311,
        transport="tcp"
    )

    # Assign callback functions
    client.on_connect = on_connect
    client.on_message = on_message
    client.on_subscribe = on_subscribe

    # Configure authentication if needed
    client.username_pw_set(username=None, password=None)

    # Connect to the broker
    client.connect(THE_BROKER, port=1883, keepalive=60)

    # Blocking call to process network traffic, dispatch callbacks,
    and handle reconnection
    client.loop_forever()
```

Python example 1: output

```
$ python3 sisub200.py
Connected to test.mosquitto.org:1883
Flags: ConnectFlags(session_present=False)
Reason Code: Success
Properties: []
Subscribed: 1 [ReasonCode(Suback, 'Granted QoS 0')]
$SYS/broker/connection/7bcb08e5fe83.connection-to-external-test-broker/state 0 b'1'
$SYS/broker/connection/Cinder.Public_Bridge/state 0 b'1'
$SYS/broker/connection/DESKTOP-N6DSFNQ.connection-bridge-test/state 0 b'1'
$SYS/broker/connection/Home_Assistant_Bridge_Growatt-25KTL3/state 0 b'1'
$SYS/broker/connection/LORA_CHOST/state 0 b'1'
$SYS/broker/connection/Lange-Chirp.test-mosquitto-org/state 0 b'1'
$SYS/broker/connection/MQTT-Broker.Cloud/state 0 b'1'
$SYS/broker/connection/OLIROIIA.broliro/state 0 b'1'
$SYS/broker/connection/PADAVAN2/state 0 b'1'
$SYS/broker/connection/SHIRE.OT-to-IT/state 0 b'1'
$SYS/broker/connection/Win11-2022TCRCW.broker2/state 0 b'1'
...
```

Python example 2: very basic periodic producer

```
# File: sipub.py
#
# The simplest MQTT producer using paho-mqtt (version 2.0.0)
```

```
import random
import time
import paho.mqtt.client as mqtt
```

Configuration

```
THE_BROKER = "test.mosquitto.org"
THE_TOPIC = "PMtest/rndvalue"
```

```
# Callback function triggered when the client receives a CONNACK
response from the broker.
```

```
def on_connect(mqttc, obj, flags, reason_code, properties):
    print(f"Connected to {mqttc_host} port: {mqttc_port}")
    print(f"Flags: {flags}")
    print(f"Reason Code: {reason_code}")
    print(f"Properties: {properties}")
```

```
# Callback function triggered when a message is published.
```

```
def on_publish(mqttc, obj, mid, reason_code, properties):
    print(f"Message ID: {mid}, Reason Code: {reason_code}")
```

```
if __name__ == "__main__":
```

```
    # Create a new MQTT client instance
```

```
    client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2,
        client_id="",
        clean_session=True,
        userdata=None,
        protocol=mqtt.MQTTv311,
        transport="tcp",
    )
```

```
# Assign callback functions
```

```
client.on_connect = on_connect
client.on_publish = on_publish
```

```
# Set username and password if required
```

```
client.username_pw_set(username=None, password=None)
```

```
# Connect to the MQTT broker
```

```
client.connect(THE_BROKER, port=1883, keepalive=60)
```

```
try:
```

```
    while True:
```

```
        # Generate a random value and publish it to the topic
```

```
        random_value = random.uniform(0, 100)
```

```
        result = client.publish(THE_TOPIC, payload=f"{random_value:.2f}", qos=0, retain=False)
```

```
        print(f"Published: {random_value:.2f} to topic {THE_TOPIC}")
```

```
        # Wait before sending the next value
```

```
        time.sleep(5)
```

```
except KeyboardInterrupt:
```

```
    print("Exiting...")
```

```
    client.disconnect()
```

Warm-up Task



■ Exercise #1

- Split the class in two: half of the class will publish data using their smartphones and the other half will subscribe using smartphones or MQTT software
- You must first agree on:
 - The broker you will use
 - The topic you will use to publish/subscribe

■ Exercise #2

- Download **phyphox** (Android and iOS, free)
- Reads accelerometer, gyroscope, magnetometer, light, pressure, GPS
- MQTT goes in a custom experiment file (.phyphox): a network connection with service **mqtt/json**, broker, topic and values to send
- Load the file (+ button or QR code), start it, subscribe to the topic
- Guide: phyphox.org/wiki, “Network” in the file format page
- Experiment with the app and send data from your phone’s sensors to an MQTT broker.
- Can you see the data on an MQTT subscriber?

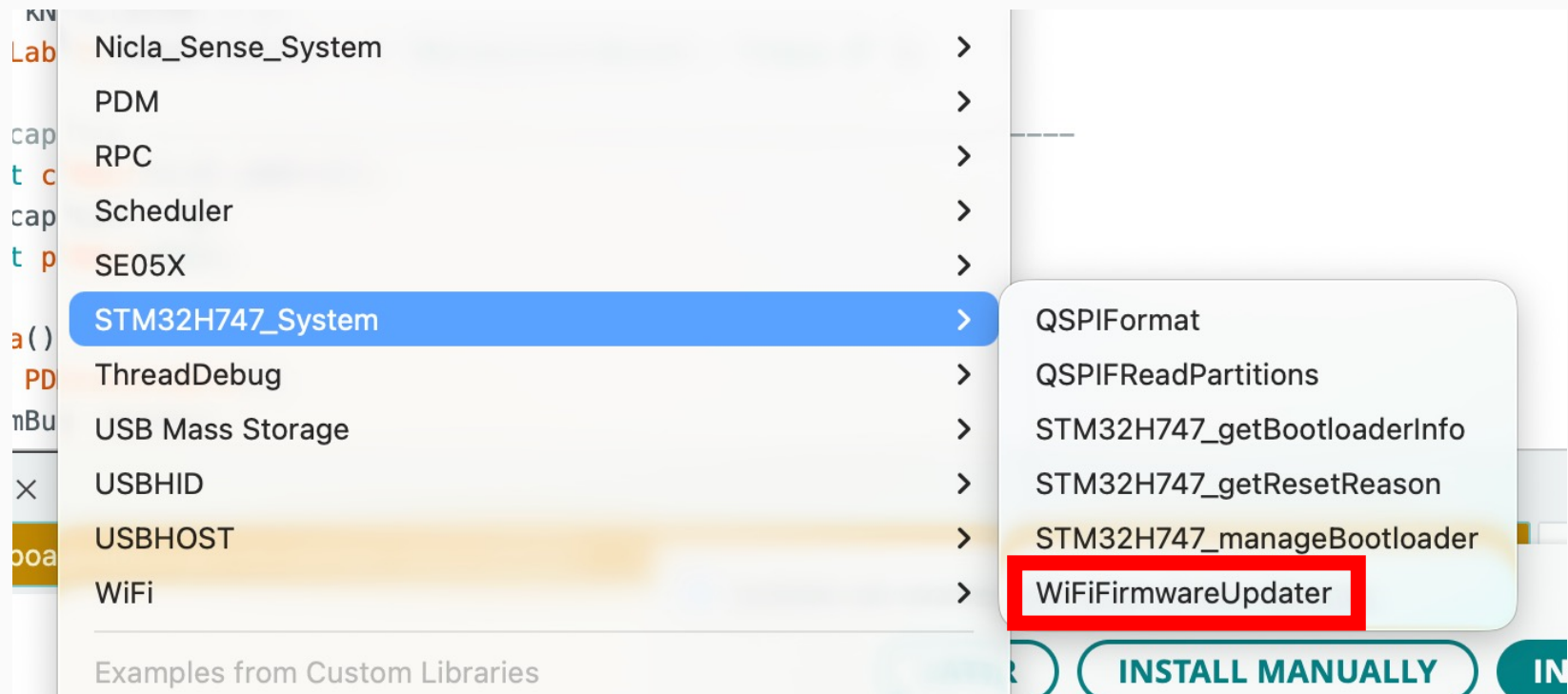
Primary Task



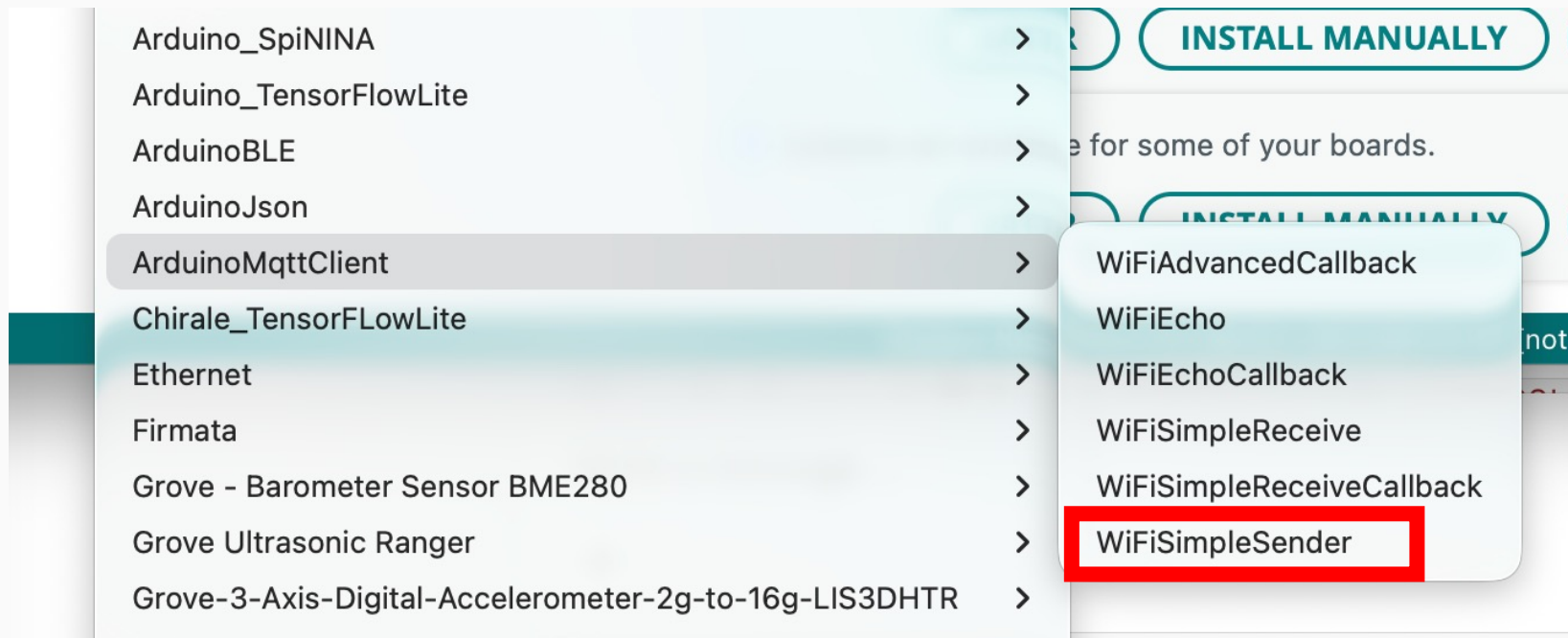
Set Up MQTT on the Nicla Vision

- **Install the board core:** Tools → Board → Boards Manager, search “Arduino Mbed OS Nicla Boards” and install it, then select Arduino Nicla Vision as your board.
- **Add the MQTT library:** Tools → Manage Libraries, search “ArduinoMqttClient” (by Arduino) and click Install — it is the official MQTT library and pulls in everything you need.
- **Gather your connection details:** your Wi-Fi network name (SSID) and password, plus an MQTT broker address such as `broker.hivemq.com` on port 1883.

Set Up MQTT on the Nicla Vision



Set Up MQTT on the Nicla Vision



Connect to Wi-Fi and the MQTT Broker

```
#include <WiFi.h>
#include <ArduinoMqttClient.h>

const char* ssid = "YOUR_SSID";
const char* pass = "YOUR_PASSWORD";
const char broker[] = "broker.hivemq.com";
const int port = 1883;

WiFiClient net;
MqttClient mqttClient(net);

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, pass);
  while (WiFi.status() != WL_CONNECTED) delay(500);
  while (!mqttClient.connect(broker, port)) delay(1000);
  mqttClient.subscribe("nicla/led");
}
```

Publish and Receive Messages

Register a callback with `mqttClient.onMessage(onMqttMessage)` in `setup()`. Then poll the broker and publish inside `loop()`:

```
void onMqttMessage(int size) {  
    Serial.print("Received on ");  
    Serial.println(mqttClient.messageTopic());  
}  
  
void loop() {  
    mqttClient.poll();                // process incoming  
    mqttClient.beginMessage("nicla/status");  
    mqttClient.print("online");  
    mqttClient.endMessage();  
    delay(2000);  
}
```

`mqttClient.poll()` keeps the connection alive and dispatches messages to `onMqttMessage()`.

Nicla+ MQTT

- 1) Generate a random number on the Nicla, send it via MQTT and visualize it using an MQTT client.
- 2) Send a message to the Nicla via MQTT to switch ON the LED.

Main takeaways

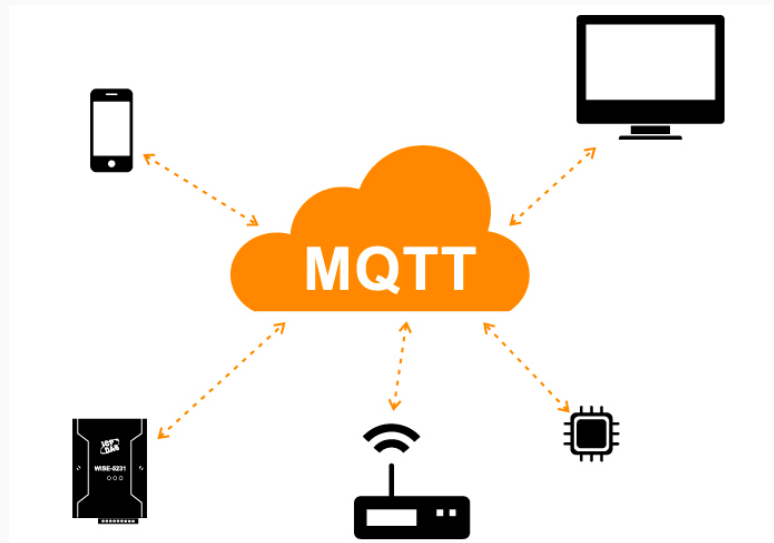
- **Publish/subscribe decouples devices:** publishers and subscribers only know the broker, which filters and routes messages by topic
- **Lightweight by design:** binary headers and low power use suit embedded devices sending many small messages over TCP or websockets
- **Topics are hierarchical:** case-sensitive paths like house/room/temp, with + and # wildcards to subscribe to whole groups
- **Reliability is configurable:** QoS 0, 1 or 2, plus retained messages, keep alive, last will and persistent sessions
- **Secure it yourself:** passwords travel in clear text, so always pair authentication with TLS (port 8883)
- **Easy to start:** Mosquitto, HiveMQ or cloud brokers, public sandboxes, and the paho-mqtt Python client
- **MQTT 5.0 evolves, not replaces:** reason codes, user properties, shared subscriptions, request-response and topic aliases

Thanks!

Thanks!

TinyML whispers low,
Code dances on micro souls,
AI blooms in small.

MQTT 5.0



Changes in the MQTT 5 Protocol

- While MQTT 5 is a major update to the existing protocol specification, the new version of the lightweight IoT protocol is more of an evolution rather than a revolution and retained all characteristics that contributed to its success
- Although some foundational mechanics were added or changed slightly, the new version still feels like MQTT and it sticks to its principles that made it the most popular Internet of Things protocol to date.
- One of the most interesting and most flexible new MQTT 5 features is the possibility to add custom key-value properties in the MQTT header. Similar to protocols like HTTP, MQTT clients and brokers can add an arbitrary number of custom (or pre-defined) headers to carry metadata. This metadata can be used for application specific data.

- Many MQTT packets now also include Reason Codes. A Reason Code indicates that a pre-defined protocol error occurred. These reason codes are typically carried on acknowledgement packets and allow client and broker to interpret error conditions (and potentially work around them). The Reason Codes are sometimes called **Negative Acknowledgements**.
- The following MQTT packets can carry Reason Codes:

- CONNACK
- PUBACK
- PUBREC
- PUBREL
- PUBCOMP
- SUBACK
- UNSUBACK
- AUTH
- DISCONNECT

User Properties

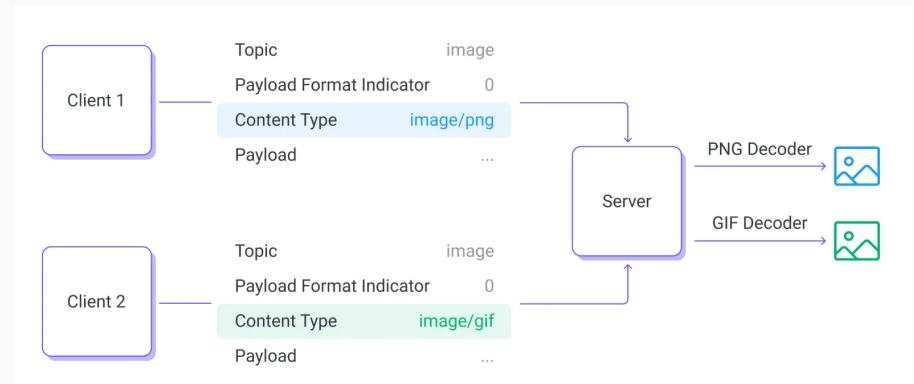
- In MQTT 5, user properties are basic UTF-8 string **key-value pairs** that you can append to almost every type of MQTT packet, excluding PINGREQ and PINGRESP. This includes control packets such as PUBREL, or PUBCOMP.
- As long as you don't exceed the maximum message size (256MB), you can use an unlimited number of user properties to add metadata to MQTT messages and pass information between publisher, broker, and subscriber.
- **The feature is very similar to the HTTP header concept.**
- Why Were User Properties Introduced?
 - Two major drawbacks MQTT 3 users voiced were the lack of extensibility of the protocol and the difficulty of creating multi-vendor deployments.
 - With the added possibility to pass virtually any information across an entire MQTT system, the new specification makes sure that users can extend the standard protocol features to meet their specific needs.

Shared Subscriptions

- Shared subscriptions are an MQTT v5 feature that allows MQTT clients to share the same subscription on the broker.
- In standard MQTT subscriptions, each subscribing client receives a copy of each message that is sent to that topic. **In a shared subscription, all clients that share the same subscription in the same subscription group receive messages in an alternating fashion. This mechanism is sometimes called client load balancing, since the message load of a single topic is distributed across all subscribers.**
- Shared subscriptions use the following topic structure:
 - `$share/GROUPID/TOPIC`
- The shared subscription consists of 3 parts:
 - A static shared subscription identifier (`$share`)
 - A group identifier
 - The actual topic subscriptions (may include wildcards)
- A concrete example for such a subscriber would be:
 - `$share/my-shared-subscriber-group/myhome/groundfloor/+/temperature.`

Payload Format Description

- Modern IoT projects are often big and complex. These large-scale projects usually require the collaboration of multiple vendors and teams. Improved interoperability and transparency across systems topped the list of user requirements for MQTT version 5.
- The Payload Format Description feature aims to meet those user requirements.
- **Payload Format Indicator**
 - The Payload Format Indicator is part of any MQTT packet that can contain a payload. Specifically, a CONNECT packet that contains a WILL message or a PUBLISH packet. The indicator is an optional byte value. A value of 0 indicates an “unspecified byte stream”. A value of 1 indicates a “UTF-8 encoded payload”. If no Payload Format Indicator is provided, the default value is 0.
- **Content Type**
 - Similar to the Payload Format Indicator, the Content Type is optional and can be part of a CONNECT that contains a WILL message or of any PUBLISH. The Content Type must be a UTF-8 encoded string. The Content Type value identifies the kind of UTF-8 encoded payload. When the Payload Format Indicator is set to 1, a MIME content type descriptor is expected (but not mandatory). Any valid UTF-8 String can be used.

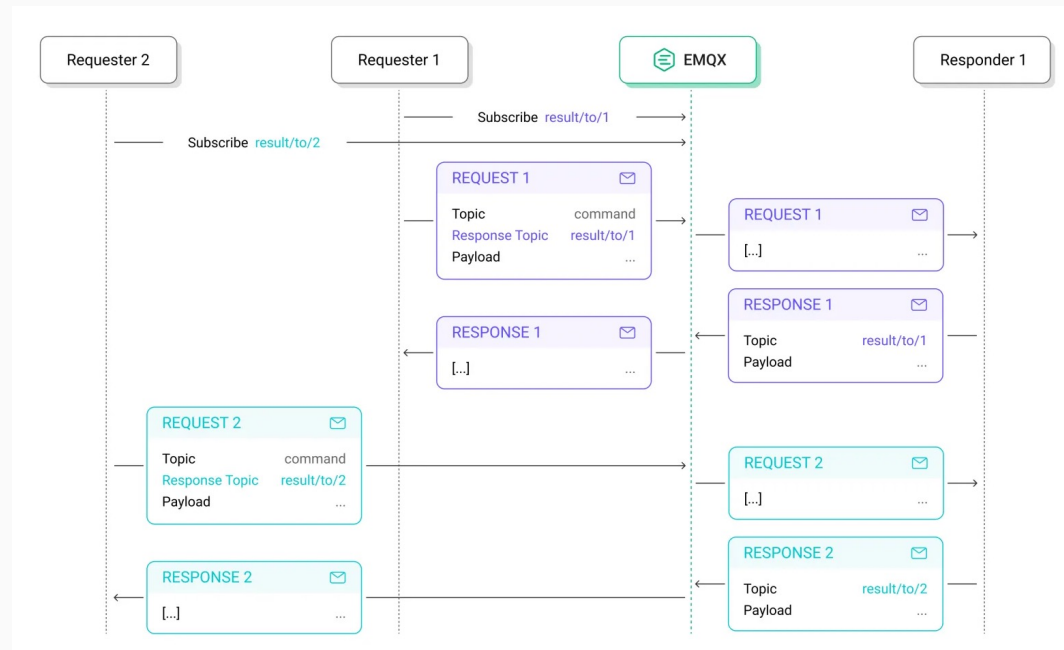


Request-Response Pattern

- The publisher-subscriber mechanism of MQTT completely decouples the sender and receiver of messages, allowing messages to be delivered asynchronously. However, this also brings a problem: even with QoS 1 and 2 messages, the publisher can only ensure that the message reaches the server, but cannot know whether the subscriber has ultimately received the message. When executing some requests or commands, the publisher may want to know the execution result of the other end.
- The most direct way is to have the subscriber return a response of the request.
- To solve these problems, MQTT 5.0 introduced properties such as Response Topic, Correlation Data, and Response Information to standardize the Request / Response pattern in MQTT.

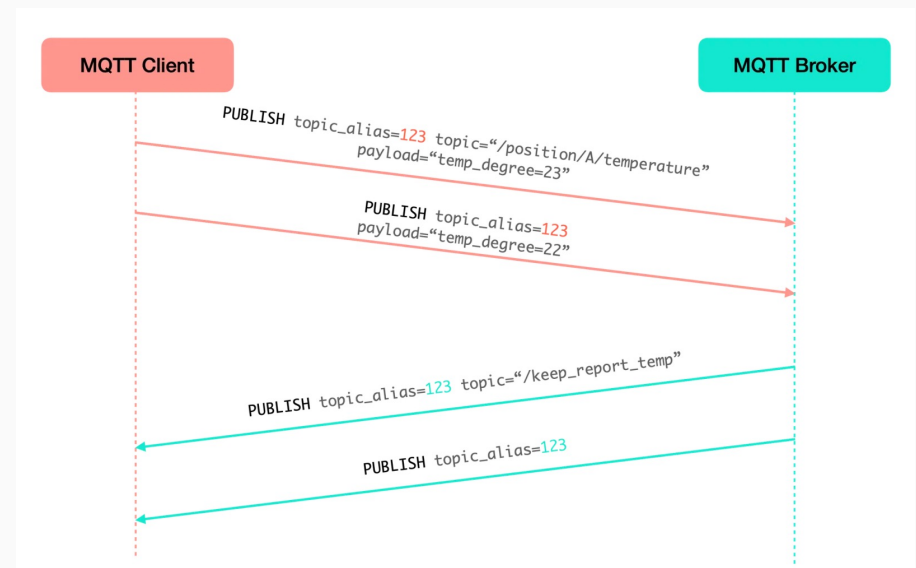
Request-Response Pattern: Response Topic

- In MQTT 5.0, the requester can specify an expected Response Topic in the request message. After taking appropriate action based on the request message, the responder publishes a response message to the Response Topic carried in the request. If the requester has subscribed to that Response Topic, it will receive the response.
- The requester can use its Client ID as part of the Response Topic, effectively avoiding conflicts caused by different requesters inadvertently using the same Response Topic.



Topic Alias

- The Topic Alias feature is specifically useful in deployments where large numbers of devices are connected and smaller messages are sent at a high frequency.
- Topic Aliases are an integer value that can be used as a substitute for topic names.
- A sender can set the Topic Alias value in the PUBLISH message, following the topic name.
- The message receiver then processes the message like any other PUBLISH and persists a mapping between the Integer (Topic Alias) and String (Topic Name).
- Any following PUBLISH message for the same topic name can then be sent with an empty topic name, only using the defined Topic Alias.



Topic Alias use case

- In a number of use cases, such as predictive maintenance, the quality and responsiveness of the service can be improved by sending small data points in real time. Implementing a use case like this, the deep and individualized topic name may exceed the size of the actual payload, that may be represented by a single integer value. For example a topic name like
 - `data/europe/germany/south/bavaria/munich/schwabing/box-32543y/junction/ consumption/current`
- describes the current power consumption of a specific junction box. The value to be sent is a single integer.
- So when you want to send many small messages in real time with long topic names, the Topic Alias feature really shines and helps to increase performance. More importantly it reduces network traffic significantly.